System Design Document: Dead Man's Draw++

Claire Tam
Student Email: claire.tam@student.adelaide.edu.au
Student ID: a1915710@adelaide.edu.au

April 6, 2026

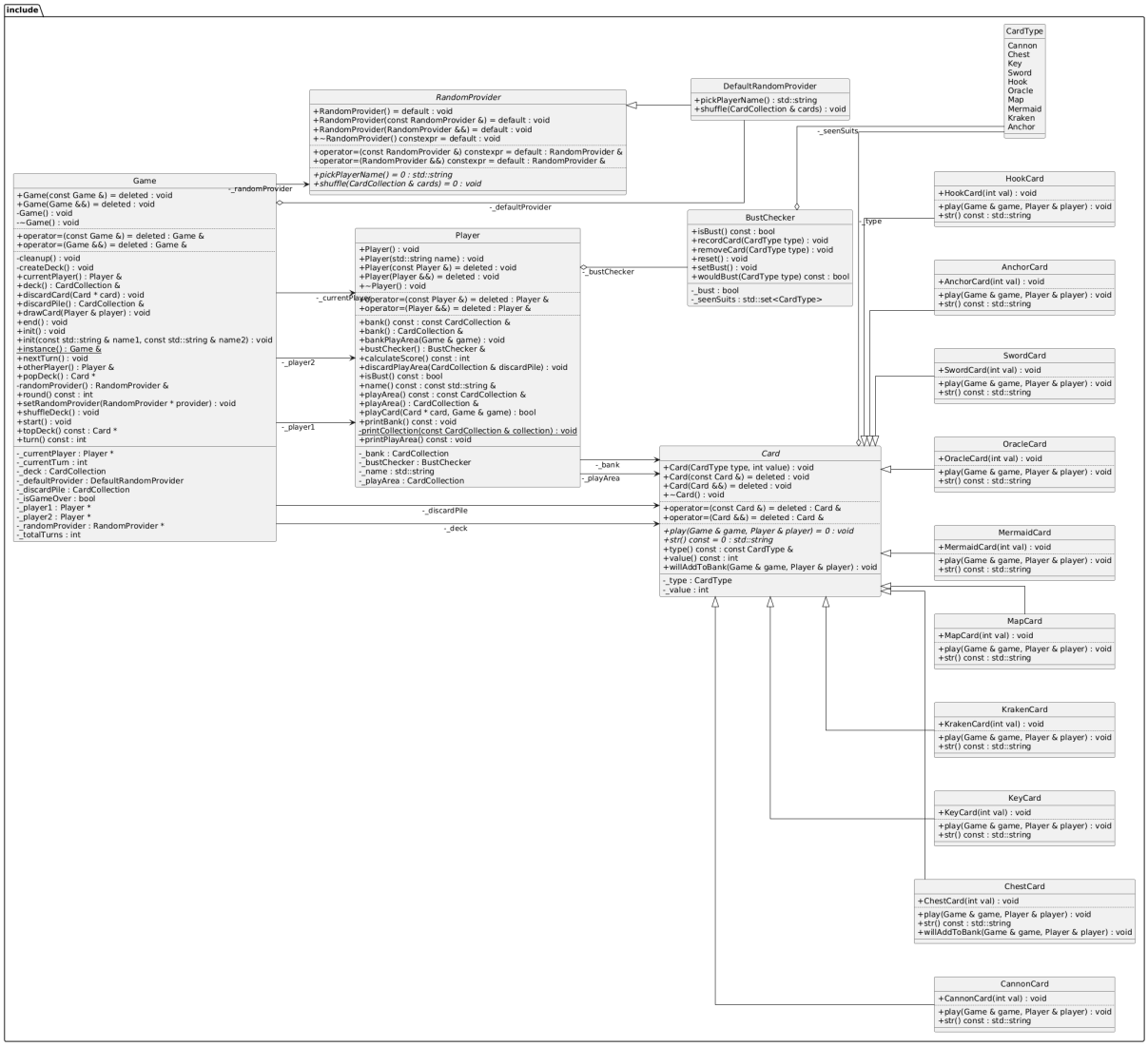
Contents

1 System Architecture Overview **2**
1.1 UML Diagram . . . 2
1.2 The `Game` Singleton . . . 2
1.3 The `Player` Entity . . . 3
1.4 The Polymorphic `Card` Interface . . . 3

2 Design Refactoring & Evolution **3**
2.1 Bust Detection Optimisation: `BustChecker` . . . 3
2.2 Separation of UI and Input: `InputHelper` & `CardSelector` . . . 3
2.3 Deterministic Testing via `RandomProvider` . . . 4
2.4 C++ Memory Management . . . 4
2.5 The `Anchor` Suit . . . 4
2.6 Const-Correctness & Performance Refinements . . . 4

1 System Architecture Overview

1.1 UML Diagram



The system is built on general Object-Oriented principles; The architecture is driven by three primary entities: `Game`, `Player`, and `Card`.

1.2 The `Game` Singleton

The `Game` class acts as the central controller and state holder. According to the rubric's ("There should only be a single instance of Game at any one time"), which implies I should use Meyer's Singleton.

- **Responsibilities**: Initialises the deck, drives the primary turn-based gameplay loop, handles overall turn/round progression, and determines the final winner.
- **Memory Ownership**: The `Game` uniquely owns the `_deck` and `_discardPile` collections. It is exclusively responsible for freeing standard non-played cards when the simulation ends.

1.3 The `Player` Entity

The `Player` class models an individual participant.

- **Responsibilities**: Maintains a transient `_playArea` collection and a persistent `_bank` collection. It computes scores based on the assignment rules (highest point value of each unique suit).
- **Memory Ownership**: `Player` takes ownership of dynamically allocated `Card` pointers from the `Game` deck once drawn. Its destructor ensures that any cards remaining in the play area or bank are successfully deleted, satisfying limits on memory leaks (DC5, DC6).

1.4 The Polymorphic `Card` Interface

The assignment dictates that card suits execute unique behaviours. By defining `Card` as an abstract base class, `Game` and `Player` are completely decoupled from suit-specific logic.

- `virtual void play(Game&, Player&) = 0;`
  Overridden by derived classes (e.g., `CannonCard`, `KrakenCard`) to execute abilities asynchronously upon being drawn.
- `virtual void willAddToBank(Game&, Player&);`
  Some abilities (specifically `ChestCard` and `KeyCard`) only trigger when banked correctly. This virtual hook allows specific cards to interrupt the banking process to draw bonus cards without cluttering the `Player::bankPlayArea` method with multiple `if (type == ...)` conditions.

2 Design Refactoring & Evolution

The system was not built via a monolithic drop; iterative refactoring was employed to refine logic and ensure the compliance of *SOLID* design principles.

2.1 Bust Detection Optimisation: `BustChecker`

Initially, checking if a player was "bust" involved iterating through the `_playArea` ($O(N)$). As the system evolved, the `BustChecker` class was extracted. By utilising a `std::set<CardType>`, the design improved bust lookup to $O(1)$ amortised complexity. This strictly compartmentalises the "state" of a turn from the actual collection of drawn cards.

2.2 Separation of UI and Input: `InputHelper` & `CardSelector`

Interactive abilities (like the `Sword` or `Cannon`) require UI prompts and input validation. Initially implemented inside the respective `Card::play()` methods, this resulted in redundant code. These were refactored out into:

- `InputHelper`: Handles loops and buffer-clearing for `std::cin`, abstracting crash hazards when users input non-numeric data strings.
- `CardSelector`: Presents the user with valid targeting options (from banks or discard piles), abstracts the menu UI, and returns the target card pointer.

This extraction aligns with the Single Responsibility Principle, ensuring `Card` derivatives focus purely on game mechanics, not user I/O.

2.3 Deterministic Testing via `RandomProvider`

To test a naturally random probability game, dependency injection was used. The `RandomProvider` interface abstracts RNG for both deck shuffling and player name selection. By explicitly passing a `NoShuffleProvider` instance during testing, deterministic deck stacking could be achieved, enhancing the GTest scenarios checking edge-cases.

2.4 C++ Memory Management

The rubric requires us to dynamic allocation using pointers (DC4) as well as safe cleanup (DC5, DC6) without relying on Smart Pointers.

- Heap allocation (`new CannonCard(...)`) happens exclusively in `Game::createDeck()`.
- Ownership transfers occur exclusively through standard vectors: moving a card to `_player.playArea()` logically abstracts ownership to the `Player`.
- Class destructors loop backwards through elements, invoking `delete card;`; Leaks can be detected using AddressSanitizer.
- Resource ownership copies are disabled by explicitly deleting copy/move assignment operators (‘`Card(const Card&) = delete;`‘, etc.).

2.5 The `Anchor` Suit

When drawn, `AnchorCard::play(...)` invokes `Player`'s `playArea` via iterators. It parses every card chronologically prior to itself, forcibly relocates them to the player's persistent `_bank`, and finally strips those `CardType` registrations from the player's `BustChecker`. This should guarantee that if a secondary drawn Anchor invalidates the current stack, the pre-Anchor cards remain the same without writing a single conditional statement inside the base `Game` loop.

2.6 Const-Correctness & Performance Refinements

- Attributes inaccessible outside their scoped lifespan are passed by `const` l-value reference.
- Querying routines (e.g., `calculateScore()`, `isBust()`) are strictly suffixed with `const`.
- Compilers are hinted to restrict memory footprint optimisations; prominently, the universal `CardType` enumeration was constrained to use a 1-byte data structure representation (‘`enum class CardType : std::uint8_t`‘).
- Magic strings/values have been extracted to `constexpr` headers (such as `GAME_TITLE`).